



**UNIVERSIDAD
DE GRANADA**



Práctica 3 - Desarrollo de Software

Claudio Rivas Boza
Aitor Fernández Pérez
Gerardo Pérez Triviño

May 3, 2026

1 Introducción y Objetivos

En esta práctica desarrollaremos un sistema de gestión de paquetes vacacionales para una plataforma de reservas, aplicando los patrones de diseño *Composite* y *Strategy*, e implementando un conjunto de pruebas unitarias que verifiquen la integridad del sistema.

1.1 Objetivos generales

- Analizar el diagrama de clases UML para realizar el ejercicio grupal.
- Diseñar pruebas para detectar el mayor número de errores.
- Aprender a hacer pruebas de unidad en Flutter/Dart y de componentes en Flutter.
- Entender el funcionamiento de los test y groups en Flutter/Dart.

2 Diseño del sistema

2.1 Descripción general

El sistema modela una plataforma de reservas de paquetes vacacionales compuesta por servicios individuales (vuelos y hoteles) que pueden agruparse de forma jerárquica en paquetes. El coste de cada servicio no depende de la política de tarificación seleccionada en tiempo de ejecución.

La arquitectura combina dos patrones de diseño:

- **Composite**: permite tratar de forma uniforme elementos individuales (*Vuelo*, *Hotel*) y agrupaciones (*Paquete*) a través de la interfaz común **ServicioTuristico**.
- **Strategy**: delegamos el cálculo del precio en objetos intercambiables (**PoliticaVuelo**, **PoliticaHotel**), lo que permite modificar el comportamiento en tiempo de ejecución sin alterar las clases de servicio.

2.2 Diagrama de clases UML

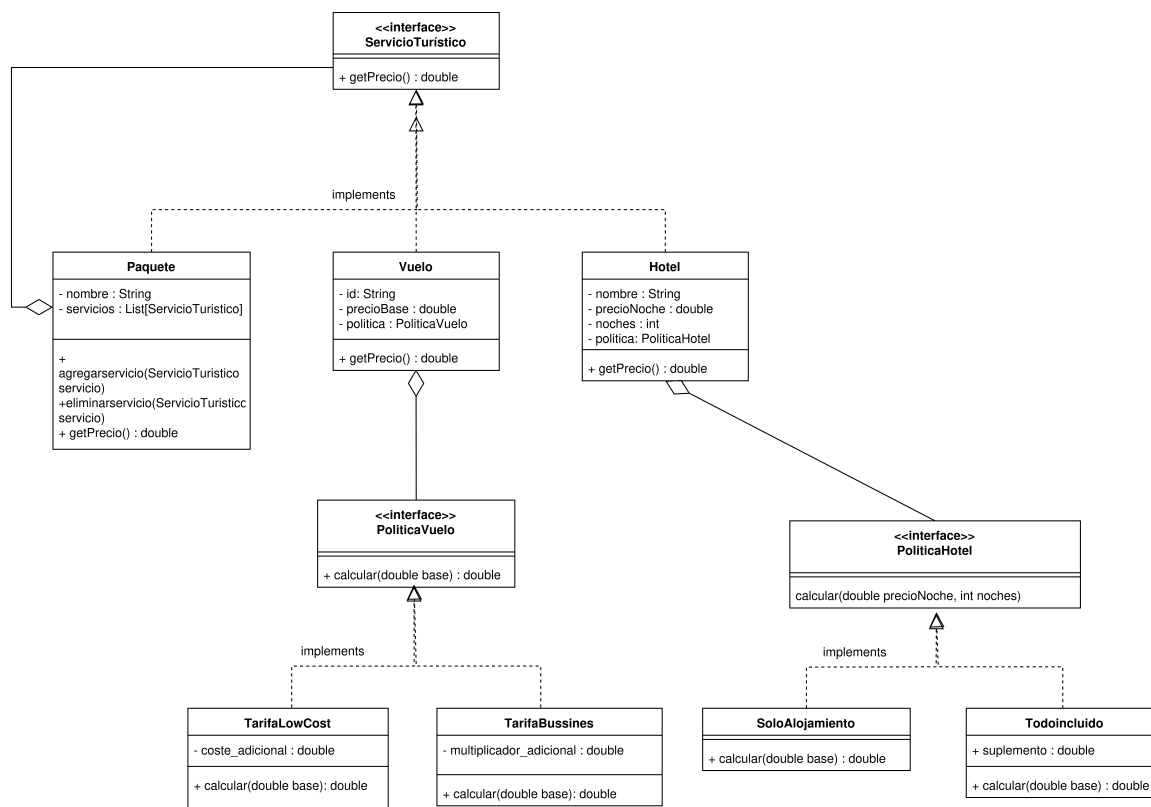


Figure 1: Diagrama de clases UML del patrón Composite y Strategy.

2.3 Descripción de las clases

2.3.1 ServicioTuristico

Interfaz raíz del patrón *Composite*. Define el método.

2.3.2 Paquete

Nodo compuesto de la jerarquía. Almacena una lista de **ServicioTuristico** y su implementación del método.

2.3.3 Vuelo

Hoja del árbol que representa un billete de avión.

2.3.4 Hotel

Hoja del árbol que representa una estancia hotelera.

2.3.5 PoliticaVuelo y sus implementaciones

- **TarifaLowCost**: aplica un recargo fijo constante al precio base.
- **TarifaBusiness**: multiplica el precio base por un factor interno.

2.3.6 PoliticaHotel y sus implementaciones

- **SoloAlojamiento:** calcula el coste como el producto de precio por noche y número de noches.
- **TodoIncluido:** añade un suplemento diario constante a cada noche.

3 Capturas del proyecto en Funcionamiento

The screenshot shows a web application titled "Paquetes Turísticos". It features a form for creating a package. At the top, there are two tabs: "Vuelo" (selected) and "Hotel". Below the tabs, there are two input fields: "ID del vuelo" with the value "1234" and "Precio base (€)" with the value "100". Below these fields, there is a dropdown menu showing "LowCost (base + 15 €)" with a downward arrow. Below the dropdown, there is a button labeled "Añadir servicio". At the bottom of the form, there is a label "Total: 0.00 €".

Figure 2: Captura 1

Paquetes Turísticos

Tipo:

Vuelo

Hotel

Nombre del hotel

Dila

Precio por noche (€)

50

Número de noches

3

Solo Alojamiento

Añadir servicio

✈️

Vuelo 1234

115.00 €

Total: 115.00 €

Figure 3: Captura 2

Paquetes Turísticos

Tipo:

Vuelo

Hotel

ID del vuelo

Bussines

Precio base (€)

100

Business (base × 3)

Añadir servicio

✈️

Vuelo 1234

115.00 €

🏨

Hotel Dila

150.00 €

✈️

Vuelo Fly Emirates 1234

450.00 €

Total: 715.00 €

Figure 4: Captura 3

4 Pruebas Unitarias

Para verificar la seguridad de implementación del sistema se han implementado 12 tests unitarios utilizando el paquete `flutter_test` de Dart, organizados en tres grupos mediante la función `group`. El primer grupo, *Políticas de Tarificación*, valida de forma aislada los cálculos matemáticos de cada política, referenciando directamente las constantes internas de cada clase en lugar de valores literales, lo que hace las pruebas robustas ante cambios en dichas constantes. El segundo grupo, *Servicios Individuales*, comprueba que los constructores de `Vuelo` y `Hotel` lanzan `ArgumentError` ante datos inválidos, y que el método `getPrecio()` de cada uno delega correctamente el cálculo en su política asignada. El tercer grupo, *Agrupación de Paquetes*, verifica el comportamiento del patrón Composite: que un paquete vacío devuelve cero, que la suma de componentes directos es aritméticamente correcta, que el anidamiento de paquetes se resuelve de forma recursiva, y que la sustitución dinámica de la política de un servicio ya contenido en un paquete se refleja inmediatamente en el precio total del paquete raíz.

5 Conclusiones

El patrón *Composite* simplifica considerablemente la gestión de estructuras jerárquicas: el `Paquete` no necesita distinguir si sus hijos son servicios simples y otros paquetes, lo que hace el sistema extensible sin modificar el código existente.

El patrón *Strategy* separa el algoritmo de cálculo, permitiendo cambiar la política en tiempo de ejecución.